Universidad Mariano Gálvez de Guatemala
Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación

LowMonitor

Diagnóstico y monitorización de hardware en ensamblador x86-64
Reporte técnico

Integrantes
- Edy Estuardo Hernández López — Módulos 1 y 2
- José Andrés Osorio Noj — Módulos 3 y 4
- Rui Luis Valdez Arévalo — Módulos 5 y 6
- Programa principal, pruebas y reporte — los tres

Catedrático: René Ornélyz
Proyecto del curso: 2026-2
Entrega: 7 de noviembre de 2026

Resumen. LowMonitor es un programa de consola hecho completamente en ensamblador x86-64 para Linux. Revisa el hardware de la computadora en seis módulos: identifica el procesador, prueba los puertos de entrada/salida, revisa la memoria RAM, mide el rendimiento del CPU, compara la velocidad de la caché contra la RAM y prueba operaciones en paralelo con SIMD. No usa librerías externas: todo se pide directamente al sistema operativo. Aquí explicamos qué hace cada módulo, qué resultados obtuvimos y qué significan.

Cómo está armado el programa

El código está dividido en un archivo principal, *lowmonitor.asm*, dos archivos de apoyo con las constantes y las rutinas para imprimir, pedir memoria y mostrar errores, y un archivo por cada módulo. Al iniciar, el programa identifica el procesador y mide su frecuencia, porque varios módulos usan esos datos. Después muestra un menú, o ejecuta directamente el módulo que se le indique al ejecutarlo.

Para compilar se usa el Makefile: make genera el ejecutable, make run abre el menú, make bench corre todos los módulos y make run-io corre el módulo de puertos con permisos de administrador. Los resultados de este reporte salen de las capturas de pantalla que están en la carpeta docs/evidencia/ del repositorio.

Equipo de pruebas	Detalle
Procesador	Intel Core i5-1235U de 12.ª generación, 2.5 GHz base
Sistema	Ubuntu 24.04 en WSL2, Linux dentro de Windows
Herramientas	NASM 2.16 y GNU ld 2.42
Resultado	Ejecutable ELF de 64 bits de 36 KB, unas 2 400 líneas de código

Módulos y resultados

Módulo 1 — Identificación del CPU Edy

Con la instrucción CPUID le preguntamos al procesador su nombre, fabricante, familia y modelo, y qué extensiones tiene. Además revisamos si el sistema operativo permite usarlas, porque no basta con que el CPU

las tenga.

```
Procesador:      12th Gen Intel(R) Core(TM) i5-1235U
Familia:         6   Modelo: 154 (0x9A)   Stepping: 4
Extensiones:     SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, AVX, AVX2, FMA
AVX-512:         no implementado en este procesador
```

El resultado coincide con lo que Intel publica para este procesador: tiene AVX2 pero no AVX-512.

Módulo 2 — Monitor de puertos E/S Edy

Leemos y escribimos en los puertos 60h del teclado, 61h del panel del sistema, 3F8h del puerto serial COM1 y 80h de diagnóstico, con tamaños de 8, 16 y 32 bits. Linux solo deja usar estos puertos con permisos de administrador. Sin ese permiso, cada intento produce un error que normalmente cerraría el programa. Por eso hicimos un manejador de errores propio: atrapa el error, lo cuenta y deja que el programa siga con la siguiente prueba.

```
Puerto 60h  (teclado)        IN  8b      : [FALLO #GP capturado -> SIGSEGV]
Puerto 80h  (POST)           OUT 8/16/32 : [FALLO #GP capturado -> SIGSEGV]
Resumen: 13 operaciones IN/OUT, 13 excepciones capturadas
```

> En nuestras pruebas con WSL2, Linux no permite el acceso a puertos, así que las 13 operaciones fallaron y las 13 fueron atrapadas sin que el programa se cayera. En un Ubuntu instalado directamente, con sudo make run-io, el módulo sí lee los valores reales de los puertos.

Módulo 3 — Diagnóstico de memoria José

Reservamos 64 MB de RAM, los llenamos con tres patrones: 0xAA, 0x55 y una secuencia que va subiendo. Después los volvemos a leer y contamos cuántos bits salieron distintos. Para estar seguros de que el detector funciona, al final cambiamos un bit a propósito y comprobamos que lo encuentre.

Patrón	Resultado	Escritura	Lectura
0xAA	PASADA, 0 bits erróneos	14.19 GB/s	8.76 GB/s
0x55	PASADA, 0 bits erróneos	12.71 GB/s	8.80 GB/s
Secuencia	PASADA, 0 bits erróneos	16.47 GB/s	10.72 GB/s
Autoprueba	1 bit cambiado → 1 bit detectado	—	—

Memoria detectada: 11.53 GB en total, 10.74 GB libres.

La memoria no tiene errores. La escritura es más rápida que la lectura porque escribimos directo a la RAM sin pasar por la caché, mientras que la lectura, además de traer los datos, tiene que compararlos.

Módulo 4 — Monitor de rendimiento del CPU José

Primero medimos la frecuencia del contador de ciclos del procesador, llamado TSC. Luego pusimos al CPU a hacer tres trabajos diferentes y contamos cuántos ciclos tardó cada uno. Con eso calculamos el CPI, ciclos por instrucción, donde menos es mejor, y el IPC, instrucciones por ciclo, donde más es mejor.

Trabajo	Ciclos	Instrucciones	CPI	IPC
A: sumas independientes	9.4 millones	80 millones	0.12	8.55
B: multiplicaciones en cadena	82.0 millones	60 millones	1.37	0.73
C: operaciones con decimales	25.9 millones	100 millones	0.26	3.87

2.50 GHz	0.49	2.05	15.44
Frecuencia	CPI promedio	IPC promedio	GFLOPS

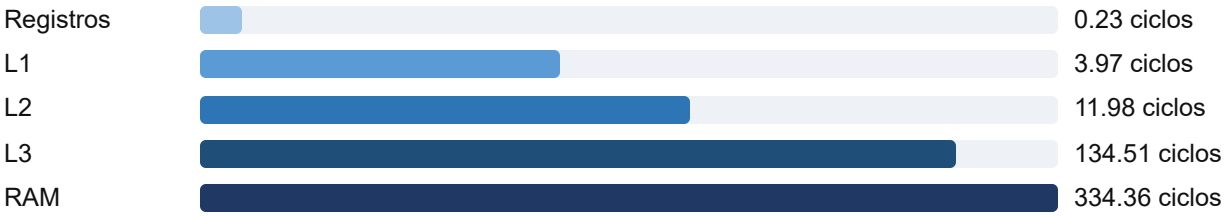
La diferencia entre A y B es la idea principal del módulo. En A las sumas no dependen unas de otras, así que el procesador hace varias al mismo tiempo. En B cada multiplicación tiene que esperar el resultado de la anterior, y el CPU no puede adelantar trabajo: por eso es unas 10 veces más lenta. El IPC de A sale en 8.55, más de lo normal, porque el procesador trabaja en modo turbo, hasta 4.4 GHz, más rápido que los 2.5 GHz del contador. Ajustando eso, el valor real es de unas 5 instrucciones por ciclo.

Módulo 5 — Gestión jerárquica de memoria Rui

Medimos cuánto tarda el procesador en obtener un dato según dónde esté: en sus registros, en la caché L1, L2 o L3, o en la RAM. Para que la medición sea justa, los datos se recorren en un orden al azar y cada lectura depende de la anterior, así el procesador no puede adivinar qué dato sigue ni traerlo antes.

Nivel	Tamaño detectado	Ciclos	Tiempo aprox.
Registros	—	0.23	0.09 ns
Caché L1	48 KB	3.97	1.6 ns
Caché L2	1.25 MB	11.98	4.8 ns
Caché L3	12 MB	134.51	53.8 ns
RAM	11.5 GB	334.36	133.7 ns

Tiempo de acceso por nivel, en escala logarítmica



Se cumple lo que vimos en clase: entre más lejos está el dato del procesador, más tarda. Cada nivel es varias veces más lento que el anterior, y leer de la RAM es unas 84 veces más lento que leer de la caché L1. Por eso la caché es tan importante para el rendimiento.

Módulo 6 — Operaciones SIMD, bonus Rui

SIMD significa hacer la misma operación sobre varios números a la vez. Sumamos a + b y lo multiplicamos por b sobre 2 048 números con tres versiones: SSE2 con 4 números a la vez, AVX2 con 8 y AVX-512 con 16. En cada caso comprobamos que el resultado sea igual al cálculo normal, uno por uno, y también hicimos una comparación a > b en paralelo.

Versión	Números a la vez	GFLOPS	Resultado	Más rápido que SSE2
SSE2	4	22.16	correcto	—
AVX2	8	34.61	correcto	1.56 veces
AVX-512	16	este procesador no lo tiene; el programa lo detecta y lo salta		

AVX2 fue 1.56 veces más rápido que SSE2. No llegó al doble porque gran parte del tiempo se va en leer y guardar los datos en memoria, no en hacer las cuentas. La comparación dio 2 045 de 2 048, que es justo lo esperado.

Optimizaciones usadas

Técnica	Para qué sirve
Escritura directa a RAM con MOVNTDQ	Llenar 64 MB rápido sin llenar la caché con datos que no se van a reusar, en el módulo 3.
Prefetch con PREFETCHT0	Pedir los datos antes de necesitarlos para no esperar a la RAM, en el módulo 3.
SIMD: SSE2, AVX2 y AVX-512	Procesar varios números en una sola instrucción, en los módulos 4 y 6.
POPCNT	Contar bits malos con una sola instrucción, en los módulos 3 y 6.
Mediciones protegidas con LFENCE y RDTSCP	Que el procesador no desordene instrucciones y altere la medición.
Macros	Reutilizar el mismo código en varias partes y que el programa sea más corto.

Organización del trabajo

Cada integrante se encargó de dos módulos, y entre los tres hicimos el programa principal, las rutinas compartidas, el Makefile, las pruebas y este reporte. Seguimos el cronograma tentativo del enunciado y cada parte se terminó en la fecha planificada.

Tarea	Responsable	Inicio	Fin
Programa principal, rutinas compartidas y Makefile	Los tres	06/09	12/09
Módulo 1 — Identificación del CPU	Edy	13/09	19/09
Módulo 2 — Monitor de puertos E/S	Edy	20/09	26/09
Módulo 3 — Diagnóstico de memoria	José	04/10	10/10
Módulo 4 — Monitor de rendimiento del CPU	José	11/10	17/10
Módulo 5 — Gestión jerárquica de memoria	Rui	25/10	31/10
Módulo 6 — Operaciones SIMD, bonus	Rui	01/11	07/11
Pruebas, reporte técnico y entrega	Los tres	01/11	07/11

El detalle completo y el diagrama de Gantt están en docs/Cronograma_LowMonitor.xlsx.

Limitaciones y conclusiones

- Puertos E/S: necesitan permisos de administrador. Sin ellos el programa no se cae; muestra el error y sigue.
- Contadores del procesador: Ubuntu los bloquea para usuarios normales, así que medimos con el contador de ciclos TSC. Por el modo turbo algunos IPC salen más altos de lo real.
- WSL2: las pruebas se hicieron en Linux dentro de Windows, que agrega algo de ruido a las mediciones de L3 y RAM.
- AVX-512: está programado, pero no se pudo probar porque este procesador no lo tiene.

LowMonitor cumple los seis módulos del proyecto usando solo ensamblador x86-64 de 64 bits y llamadas directas al sistema operativo. Con este proyecto pudimos ver con números reales tres ideas del curso: el procesador es mucho más rápido cuando las instrucciones no dependen unas de otras; la caché es unas 84 veces más rápida que la RAM; y hacer operaciones en paralelo con SIMD acelera el programa, aunque la memoria pone un límite.

Referencias

Páginas web

Chapman, R. A. (s. f.). *Linux system call table for x86 64*. https://blog.rchapman.org/posts/Linux_System_Call_Table_for_x86_64/

Cloutier, F. (s. f.). *x86 and amd64 instruction reference*. https://www.felixcloutier.com/x86/

Drepper, U. (2007). *What every programmer should know about memory*. LWN.net. https://lwn.net/Articles/250967/

Fog, A. (s. f.). *Software optimization resources*. https://www.agner.org/optimize/

Intel Corporation. (2024). *Intel® 64 and IA-32 architectures software developer's manuals*. https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html

Kerrisk, M. (s. f.). *ioperm(2) — Linux manual page*. man7.org. https://man7.org/linux/man-pages/man2/ioperm.2.html

NASM Development Team. (s. f.). *NASM — The Netwide Assembler: Documentation*. https://www.nasm.us/doc/

Repositorios de GitHub

0xAX. (2014). *asm: Learning assembly for Linux x86_64* [Repositorio de GitHub]. https://github.com/0xAX/asm

Downs, T. (2017). *uarch-bench: A benchmark for low-level CPU micro-architectural features* [Repositorio de GitHub]. https://github.com/travisdowns/uarch-bench

Memtest86+ Team. (2022). *memtest86plus: Official repo for Memtest86+* [Repositorio de GitHub]. https://github.com/memtest86plus/memtest86plus

Noonan, S. (2011). *cpuid: A simple CPUID decoder/dumper for x86/x86_64* [Repositorio de GitHub]. https://github.com/tycho/cpuid

Santilli, C. (2012). *x86-assembly-cheat: x86 IA-32 and x86-64 userland minimal examples* [Repositorio de GitHub]. https://github.com/cirosantilli/x86-assembly-cheat

Siamashka, S. (2012). *tinymembench: Simple benchmark for memory throughput and latency* [Repositorio de GitHub]. https://github.com/ssvb/tinymembench

Torvalds, L. (s. f.). *syscall_64.tbl* [Archivo del repositorio linux en GitHub]. https://github.com/torvalds/linux/blob/master/arch/x86/entry/syscalls/syscall_64.tbl